

Building a REST API Server with PHP

| Part 1: JSON Storage

Example Project: Building a Student Management API Server

- GET /students - Get all students
- GET /students/{id} - Get student by ID
- POST /students - Create new student
- PUT /students/{id} - Update student
- DELETE /students/{id} - Delete student

See It Work First

Start the JSON version of the server from the `api/` directory:

```
cd api  
php -S localhost:8080
```

Then request the list of students:

```
curl http://localhost:8080/students
```

By default, `./start_server.sh` automatically resets the storage to start from a clean state (`{ "success": true, "data": [], "count": 0 }`).

- If you want to preserve existing records across restarts, use `./start_server.sh --no-reset`.
- Either way, everything on the following slides explains how this request travels through `index.php` to produce the response.

This is an example project, so we could add more endpoints, for example:

- `GET /students?name=Kim` - search students whose name contains "Kim"

A search is still a **GET** on an existing resource — we add a query string (**?name=...**) instead of inventing a new endpoint.

- **GET** is the right choice for any read-only request.
- In this project, **POST** is what we use to create new resources (an HTML **<form>** can actually use either **GET** or **POST**).

Project Structure

```
api/
├── index.php           # Main entry point and routing
├── handlers.php        # JSON handler functions
├── handlers_pdo.php    # MySQL/PDO handler functions
├── database.php        # PDO connection
├── schema.sql          # MySQL schema for the REST API
├── models/Student.php  # Student model class for the JSON version
├── data/students.json  # JSON data storage (created automatically; not in the repo)
├── student_api_test.html # Web testing interface
├── ensure_install_packages.sh # Installs/starts PHP & MySQL if missing
├── ensure_delete_packages.sh # Recovery only: removes PHP & MySQL
├── reset_db.sh         # Database reset script
├── start_server.sh     # Server startup script
└── stop_server.sh      # Server & MySQL shutdown script
```

Shell Scripts

`start_server.sh`, `reset_db.sh`, and `stop_server.sh` all `source` `ensure_install_packages.sh`, so the first time you run any of them, PHP and MySQL are installed and started automatically if they aren't already — you don't install anything by hand first.

- Full details, options, and the recovery-only `ensure_delete_packages.sh`: see `0. Shell Script for Automation.md`.

| Database Usage

Two Databases:

- In this lecture we use JSON (`data/students.json`) so that storage setup does not hide the REST concepts.
- Once the router and HTTP methods work end to end, the next lecture switches only the handlers to PDO/MySQL — same URLs, same router, same tests.

This reuses the Week 3 database pattern and serves as a bridge to Project 1.

| Simple Test HTML

Access

`http://localhost:8080/student_api_test.htm`

`l` via HTTP while the server is running.

Warning: Never open

`student_api_test.html` directly as a

`file://` URL in the browser — browser

security will block all API requests. It must be

served through `http://localhost:8080/`.

Why We Parse the Route Manually

- `parse_url()`, `explode()`, and `$_SERVER['REQUEST_URI']` show what a router must do before it can call a handler.
- In Module 2, Laravel performs this work for us.
- We implement it once here so that Laravel routing will not feel like magic.

index.php

- Header functions
- Input parser
- Router

The code below is shown as short excerpts, grouped by topic. The full file combines them in order, with the `require` statements that load `handlers.php` (or `handlers_pdo.php`).

Header Functions

```
header('Content-Type: application/json');
```

- Declares that API responses are JSON.
- This header is sent on every request, including **OPTIONS**, which has no request body, before the router runs.

| CORS (Preview)

- If the frontend is served from a different origin, such as `http://localhost:5500` calling `http://localhost:8080`, the browser may block the request unless the API sends CORS headers such as `Access-Control-Allow-Origin`.
- CORS is a browser security rule, not a PHP rule.
- Covered in full later in this lecture.

Input Parser

- For the request

`http://localhost:8080/students/123`,

`$path` becomes `"students/123"`.

```
$method = $_SERVER['REQUEST_METHOD'];  
$path = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);  
$path = trim($path, '/');
```

Split Path Into Segments

- For the path `students/123`, we split on `/` to get `"students"` and `"123"`, stored as `$resource` and `$id`.

```
$segments = explode('/', $path);  
$resource = $segments[0] ?? ''; // students  
$id = $segments[1] ?? null;      // 123
```

- `??` (null coalescing): use the left side if it's set and not `null`, otherwise the right side — this avoids a PHP warning when `$segments[1]` doesn't exist.

Error Processing

- When the URI has no `$resource` at all, we respond with the API's own endpoint list.

```
if (empty($resource)) {  
    echo json_encode([  
        'success' => true,  
        'message' => 'Simple Student Management API',  
        'endpoints' => [  
            'GET /students' => 'Get all students',  
            'GET /students/{id}' => 'Get student by ID',  
            'POST /students' => 'Create new student',  
            'PUT /students/{id}' => 'Update student',  
            'DELETE /students/{id}' => 'Delete student'  
        ]  
    ]);  
    exit;  
}
```

Validating the Resource and the ID

Converting a string to an integer is not the same as validating it. `(int) 'abc'` becomes `0`, so an invalid ID must still be checked explicitly before use.

```
if ($resource !== 'students') {  
    http_response_code(404);  
    echo json_encode(['success' => false, 'error' => 'Resource not found']);  
    exit; // stop here  
}  
  
if (count($segments) > 2) { // reject e.g. /students/1/extra  
    http_response_code(404);  
    echo json_encode(['success' => false, 'error' => 'Resource not found']);  
    exit;  
}
```

```

$student_id = null;
if (isset($segments[1]) && $segments[1] !== '') {
    if (!ctype_digit($segments[1])) { // reject "abc", "12abc"
        http_response_code(400);
        echo json_encode(['success' => false, 'error' => 'Invalid student ID']);
        exit;
    }
    $student_id = (int) $segments[1]; // the segment is digits only
}

```

- `/students/0` is a *valid* id (`$segments[1]` `=== '0'`, not empty), so it is treated as "look up student 0", not "no ID was given".

- `ctype_digit($segments[1])` is `true` only when the string is made entirely of digits (`0–9`); the `!` in front rejects anything else — `abc`, `12abc`, `-1`, `1.5`, or an empty string — as an invalid ID instead of silently casting it with `(int)`.

IMPORTANT

Router: Which Methods Does Each Path Allow?

Our two paths don't allow the same methods, so we compute the allowed set for the actual path *first*, and reject anything else with **405**.

Path	Allowed methods
/students	GET, POST
/students/{id}	GET, PUT, DELETE

```
$allowed = ($student_id !== null) ? ['GET', 'PUT', 'DELETE'] : ['GET', 'POST'];  
  
if (!in_array($method, $allowed)) {  
    header('Allow: ' . implode(', ', $allowed));  
    http_response_code(405);  
    echo json_encode(['success' => false, 'error' => 'Method not allowed']);  
    exit;  
}
```

`POST /students/1` and `PUT /students` (no id)
both correctly get `405`, with an `Allow` header
naming the methods valid for that path.

Router: Dispatching to a Handler

By the time we reach here, `$method` is guaranteed to be one of the methods allowed for this exact path.

```
switch ($method) {  
    case 'GET':  
        if ($student_id !== null) { get_student($student_id); }  
        else { get_all_students(); }  
        break;  
    case 'POST':  
        create_student();  
        break;  
    case 'PUT':  
        update_student($student_id);  
        break;  
    case 'DELETE':  
        delete_student($student_id);  
        break;  
}
```

Student.php

`models/Student.php` is a small class with a getter/setter pair for each field (`id`, `name`, `email`, `major`, `year`) and a constructor that stamps `created_at` / `updated_at`.

We only build a `Student` *object* when **creating** a record — once saved, records are read and updated as plain associative arrays. `toArray()` bridges the two shapes:

```
public function toArray() {  
    return [  
        'id' => $this->id, 'name' => $this->name, 'email' => $this->email,  
        'major' => $this->major, 'year' => $this->year,  
        'created_at' => $this->created_at, 'updated_at' => $this->updated_at  
    ];  
}
```

(See `models/Student.php` in the repo for the full getter/setter list.)

handlers.php

- Each handler is responsible for one endpoint. It sets the HTTP status code with `http_response_code()` and writes the JSON body with `echo json_encode(...)` directly — there is no separate "send response" function to look for.

HTTP Status Codes Used Here

Code	Used when
200 OK	Successful GET or PUT
201 Created	POST created a new student
400 Bad Request	Bad JSON, missing/invalid field, invalid ID
404 Not Found	Unknown resource or student ID
405 Method Not Allowed	Wrong method for this path, e.g. POST /students/1
500 Server Error	Data file could not be read, parsed, or saved

| Utility Functions

- A file that can't be read or parsed as a JSON list (e.g. `{"id":1}` instead of `[...]`) is treated as **corrupted**, not "no students yet".

```

function load_students() {
    $file_path = 'data/students.json';

    if (!file_exists($file_path)) {
        return [];
    }

    $json_data = file_get_contents($file_path);
    if ($json_data === false) {
        http_response_code(500);
        echo json_encode(['success' => false, 'error' => 'Could not read student data file']);
        exit;
    }

    $students = json_decode($json_data, true);
    if (!is_array($students) || !array_is_list($students)) { // array_is_list(): PHP 8.1+
        http_response_code(500);
        echo json_encode(['success' => false, 'error' => 'Student data file is corrupted']);
        exit;
    }

    return $students;
}

```

- Save students to the JSON file. Both `json_encode()` and `file_put_contents()` can fail, so we check both.

```
function save_students($students) {  
    $file_path = 'data/students.json';  
    $json_data = json_encode($students, JSON_PRETTY_PRINT);  
    if ($json_data === false) {  
        return false; // e.g. invalid (non-UTF-8) data in a field  
    }  
    $bytes_written = file_put_contents($file_path, $json_data);  
    return $bytes_written !== false;  
}
```

A missing folder or a permissions problem makes `file_put_contents()` return `false`. If we ignore that, the handler can still report success (`201 Created`) even though nothing was saved.

- Get the next available ID.

```
function get_next_id($students) {  
    $max_id = 0;  
    foreach ($students as $student) {  
        if ($student['id'] > $max_id) {  
            $max_id = $student['id'];  
        }  
    }  
    return $max_id + 1;  
}
```

`max_id + 1` can reuse a deleted ID or collide under concurrent requests — suitable only for this single-user exercise (a real database fixes this, as we'll see in the next lecture)

- Reads the raw body and tells us whether it was a JSON **object** (`{...}`), not an array, string, or number.

```
function getRequestData()
{
    $raw = file_get_contents('php://input');
    $decoded = json_decode($raw);           // object mode first
    $is_object = is_object($decoded);
    $data = $is_object ? json_decode($raw, true) : null;
    return [$data, $is_object];
}
```

Reads `php://input`, not `$_POST` — `$_POST` only populates for classic HTML form submissions.

IMPORTANT

- One validation function, shared by `POST` and `PUT`, so a record can never be saved with an empty name, a malformed email, or a `major` or `year` of the wrong type — no matter which endpoint created it.

```
function validate_student_fields($name, $email, $major, $year) {  
    if (!is_string($name) || trim($name) === '') {  
        return 'name must be a non-empty string';  
    }  
    if (!is_string($email) || !filter_var($email, FILTER_VALIDATE_EMAIL)) {  
        return 'email is not a valid email address';  
    }  
    if (!is_string($major)) {  
        return 'major must be a string';  
    }  
    if (!is_int($year) || $year < 1 || $year > 6) {  
        return 'year must be an integer from 1 to 6';  
    }  
    return null; // valid  
}
```

Handlers

| GET /Students

Get all students

```
function get_all_students() {  
    $students = load_students();  
    echo json_encode([  
        'success' => true,  
        'data' => $students,  
        'count' => count($students)  
    ]);  
}
```

GET /students/{id}

Get student by ID

```
function get_student($id) {  
    $students = load_students();  
  
    foreach ($students as $student) {  
        if ($student['id'] === $id) {  
            echo json_encode(['success' => true, 'data' => $student]);  
            return;  
        }  
    }  
  
    http_response_code(404);  
    echo json_encode(['success' => false, 'error' => 'Student not found']);  
}
```

| POST /students

Create new student.

- There are four steps for this method.

Step 1: check the request shape

A valid create request needs to pass three checks, in order: **is it valid JSON at all? is it a JSON object? do the required fields have acceptable values?**

```
function create_student() {  
    [$input, $is_object] = getRequestData();  
  
    // Step 1: must be valid JSON, and specifically a JSON object – not e.g. "5" or [1,2]  
    if (!$is_object) {  
        http_response_code(400);  
        echo json_encode(['success' => false, 'error' => 'Body must be a valid JSON object']);  
        return;  
    }  
}
```

Step 2: required fields

```
// Step 2: name and email must be present (their values are checked next)
if (!array_key_exists('name', $input) || !array_key_exists('email', $input)) {
    http_response_code(400);
    echo json_encode(['success' => false, 'error' => 'name and email are required']);
    return;
}
```

Step 3: field values (same rules as **PUT** uses)

```
// Step 3: validate the actual values – shared with update_student()
$major = $input['major'] ?? '';
$year = $input['year'] ?? 1;
$error = validate_student_fields($input['name'], $input['email'], $major, $year);
if ($error !== null) {
    http_response_code(400);
    echo json_encode(['success' => false, 'error' => $error]);
    return;
}
```

{} fails at Step 2 (missing **name** / **email**);

{"name":"","email":"x"} fails at Step 3

instead of silently creating a near-empty student.

Step 4: build and save the new student

```
$students = load_students();
$new_id = get_next_id($students);

$new_student = new Student();
$new_student->setId($new_id);
$new_student->setName($input['name']);
$new_student->setEmail($input['email']);
$new_student->setMajor($major);
$new_student->setYear($year);

$students[] = $new_student->toArray();

if (!save_students($students)) {
    http_response_code(500);
    echo json_encode(['success' => false, 'error' => 'Could not save student data']);
    return;
}

http_response_code(201);
echo json_encode([
    'success' => true,
    'message' => 'Student created successfully',
    'data' => $new_student->toArray()
]);
}
```

| PUT /students/{id}

Update student

PUT replaces the whole resource, so every field is required — validated with the same `validate_student_fields()` that **POST** uses.

```
function update_student($id)
{
    [$input, $is_object] = getRequestData();
    if (!$is_object) {
        http_response_code(400);
        echo json_encode(['success' => false, 'error' => 'Body must be a valid JSON object']);
        return;
    }
}
```

- Every field must be present, and the values go through the same check `create_student()` uses — a `PUT` can't save data a `POST` would reject.

```
foreach (['name', 'email', 'major', 'year'] as $field) {  
    if (!array_key_exists($field, $input)) {  
        http_response_code(400);  
        echo json_encode(['success' => false, 'error' => "Missing required field: $field"]);  
        return;  
    }  
}  
  
$error = validate_student_fields($input['name'], $input['email'], $input['major'], $input['year']);  
if ($error !== null) {  
    http_response_code(400);  
    echo json_encode(['success' => false, 'error' => $error]);  
    return;  
}
```

```

$students = load_students();

foreach ($students as $i => $student) {
    if ($student['id'] === $id) {
        $students[$i]['name']      = trim($input['name']);
        $students[$i]['email']     = trim($input['email']);
        $students[$i]['major']     = trim($input['major']);
        $students[$i]['year']      = $input['year'];
        $students[$i]['updated_at'] = date('Y-m-d H:i:s');

        if (!save_students($students)) {
            http_response_code(500);
            echo json_encode(['success' => false, 'error' => 'Could not save student data']);
            return;
        }
        echo json_encode([
            'success' => true,
            'message' => 'Student updated successfully',
            'data' => $students[$i]
        ]);
        return;
    }
}

http_response_code(404);
echo json_encode(['success' => false, 'error' => 'Student not found']);
}

```

DELETE /students/{id}

Delete student

```
function delete_student($id)
{
    $students = load_students();

    foreach ($students as $i => $student) {
        if ($student['id'] === $id) {
            $deleted_student = $student;
            array_splice($students, $i, 1);

            if (!save_students($students)) {
                http_response_code(500);
                echo json_encode(['success' => false, 'error' => 'Could not save student data']);
                return;
            }
            echo json_encode([
                'success' => true,
                'message' => 'Student deleted successfully',
                'data' => $deleted_student
            ]);
            return;
        }
    }

    http_response_code(404);
    echo json_encode(['success' => false, 'error' => 'Student not found']);
}
```

- `array_splice($students, $i, 1)` removes and re-indexes in one step — `unset()` alone would leave a gap in the numeric keys.

Testing Failure Cases

Request	Status	Why
GET /teachers	404	Unknown resource
GET /students/abc	400	ID is not numeric
POST /students/1	405	/students/{id} doesn't allow POST
POST /students with {}	400	Missing required fields
PUT ... with {"name":"","email":"bad",...}	400	Field values fail validation

The Testing lecture walks through a complete success-and-failure test suite for these same endpoints, including the `Allow` header check for `405`.

Cross-Origin Requests (CORS)

- CORS applies once a frontend on one **origin** (scheme + host + port) calls this API from a *different* origin — e.g. a React dev server on `http://localhost:3000` calling `http://localhost:8080`.
- Same-origin requests (like our own `student_api_test.html`, served from this same API) are never subject to CORS at all.

CORS Headers

```
header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS');  
header('Access-Control-Allow-Headers: Content-Type, Authorization');  
header('Access-Control-Allow-Origin: *');
```

- **Allow-Headers** must list every custom header a cross-origin request sends (e.g. **Authorization** for a Bearer token), or the preflight fails.
- **Allow-Origin: *** does **not** work for a *credentialed* request (cookies via **credentials: 'include'**) — that needs a specific origin plus **Access-Control-Allow-Credentials: true**.

| The Preflight Request

- The browser sends this request first to check whether the server allows the actual cross-origin request.
- Without this check, the browser may block the actual cross-origin request for security.

```
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {  
    http_response_code(200);  
    exit();  
}
```

- This check runs before the router, at the top of `index.php`.

- Browsers automatically send this **OPTIONS** request first for a cross-origin, **non-simple** request — **PUT** / **DELETE**, a custom header, or a JSON body (this includes our JSON **POST**s).
- Simple requests (**GET** / **POST** / **HEAD** with plain form-style bodies) skip the preflight.

Coming Next

We now have a complete, working JSON-backed REST API, including CORS support for a separate frontend.

- The next lecture keeps every URL, method, and status code exactly the same, and replaces only the storage layer with MySQL via PDO.